

Lab 4: VibeCoding - Chat History

Pac-Man (Pygame) bug fix and feature additions | Suhas (PES1UG24AM451) | Tool: Claude

1. Understand the code structure

Prompt

I'm working on a single-file Pac-Man clone in Pygame. Here is the full game.py: [full game.py pasted]. Don't change anything yet. Briefly summarize the structure: the classes, where eat(), update(), and draw() live, and the three stub functions ghost_color, on_pellet_eaten, bonus_life_threshold.

Reply (summary)

Summarized the file: module-level constants and helpers; a **Ghost** class (reset, target, options, step, reverse); a **Game** class holding eat(), check_collisions(), update() and draw(); and the main() loop.

Stubs: ghost_color() is used in draw() as *ghost_color(...)* or *color*, so None keeps the default;

on_pellet_eaten() is called at the end of eat(); bonus_life_threshold() is read every frame in update(), and None disables bonus lives.

Flagged a bug: eat() checks for uppercase "O", but the maze uses lowercase "o" for power pellets.

2. Fix the power-pellet bug

Prompt

Bug: eating a power pellet only gives 10 points and frightened mode never triggers. I suspect eat() checks the cell against a different character than the one the maze grid uses for power pellets. Find the mismatch, explain it in 1-2 lines, and give me only the corrected lines with surrounding context. Make the minimal fix.

Reply (summary)

eat() compared against "O", while MAZE uses "o". The check never matched, so the 40-point bonus and fright mode never ran. Changing one character fixes it: a power pellet now gives 50 points, starts fright mode and reverses the ghosts.

```
self.pellets.remove(cell)
self.score += 10
if MAZE[cell[0]][cell[1]] == "o":
    self.score += 40
    self.fright_left = FRIGHT_SECONDS
```

3. Feature: ghost_color()

Prompt

Implement ghost_color(name, mode). It's called as color = ghost_color(ghost.name, mode) or color. In "frightened" mode, give each ghost a distinct tint, clearly different from the default blue. Return None for other modes. Then give me the complete updated game.py.

Reply (summary)

Added a FRIGHT_TINTS table (Blinky violet, Pinky green, Inky magenta, Clyde brown) and returned the full updated file.

Noted the tradeoff: the tints also override the white warning flash in the last second of fright mode.

```
FRIGHT_TINTS = {
    "blinky": (160, 90, 255),    # violet
    "pinky": (80, 220, 120),     # green
    "inky": (230, 70, 200),      # magenta
    "clyde": (170, 110, 60),     # brown
}

def ghost_color(name, mode):
    if mode == "frightened":
        return FRIGHT_TINTS.get(name)
    return None
```

4. Feature: on_pellet_eaten()

Prompt

Implement on_pellet_eaten(score, pellets_left). Its return value is ignored and it has no access to self, so use a small module-level dict for state that draw() reads. Show a short on-screen message like "25 pellets left!" every time pellets_left is a multiple of 25, displayed for about 1.5 seconds using pygame.time.get_ticks(). Show me exactly which lines to add or replace, and where.

Reply (summary)

Replaced the stub and added a module-level PELLET_MSG dict. Added three lines in Game.draw() right after the HUD blit, so the message appears right-aligned in the HUD bar for 1.5 s. The value 0 is skipped because the win screen shows at that point.

```
PELLET_MSG = {"text": None, "until": 0}
PELLET_MSG_MS = 1500

def on_pellet_eaten(score, pellets_left):
    if pellets_left > 0 and pellets_left % 25 == 0:
        PELLET_MSG["text"] = f"{pellets_left} pellets left!"
        PELLET_MSG["until"] = pygame.time.get_ticks() + PELLET_MSG_MS

# in Game.draw(), after the HUD blit:
if PELLET_MSG["text"] and pygame.time.get_ticks() < PELLET_MSG["until"]:
    msg = font.render(PELLET_MSG["text"], True, (255, 220, 120))
    screen.blit(msg, msg.get_rect(topright=(W - 8, ROWS * TILE + 6)))
```

5. Feature: `bonus_life_threshold()`

Prompt

Implement `bonus_life_threshold()`. The `self.bonus_awarded` bookkeeping in `update()` already exists. Return 500, with a comment explaining it's set low so the bonus life is demonstrable in a short demo (the README suggests 10000). Show me just the function.

Reply (summary)

The function returns 500, so `update()` awards an extra life at every multiple of 500 points.

```
def bonus_life_threshold():  
    # Deliberately low so a bonus life shows up within a short demo run.  
    # The README suggests 10000 for normal play.  
    return 500
```